# OOP

## UNIT 3 – CONSTRUCTORS

*A class protects its data. A constructor ensures that every object begins its life in a valid state.*

### LEARNING OBJECTIVES

Explain why constructors are necessary.

Describe the relationship between constructors and object lifetime.

Write default and parameterized constructors.

Explain constructor overloading.

Distinguish between object initialization and object modification.

Understand the role of the `this` pointer.

Validate constructor parameters.

Explain how constructors support encapsulation.

## WHY CONSTRUCTORS ARE NEEDED?

In the previous unit we transformed a simple `struct` into our first class.

```
class Song {
private:
    string title;
    string artist;
    int duration;
    int plays;

public:
    void print();
    void play();
    void setDuration(int seconds);
};
```

The class now protects its data through private members and public functions. Direct modification is no longer possible. . . seems like a major improvement.

But imagine we write:

```
Song song;
```

The object has been created.  What is its title?  Who is the artist? What is its duration? ….

Nothing has been initialized.

If the programmer immediately writes

```
song.print();
```

Should it display empty values? Zeros? Something else?

Encapsulation protects data from incorrect modification, but  does not guarantee that a newly created object already contains meaningful information.

An object can exist before it is actually ready to be used??  Professional software tries to avoid this situation entirely.

Every object should begin its life in a valid and predictable state.

## WHY CONSTRUCTORS EXIST

Consider the following program.

```
Song song;                // Object already exists!!
song.setTitle("Imagine");
song.setArtist("John Lennon");
song.setDuration(183);   // Only now  does it represent an actual song.
```

This design places the responsibility on the programmer:  must remember exactly which *setter* functions to call and in which order.

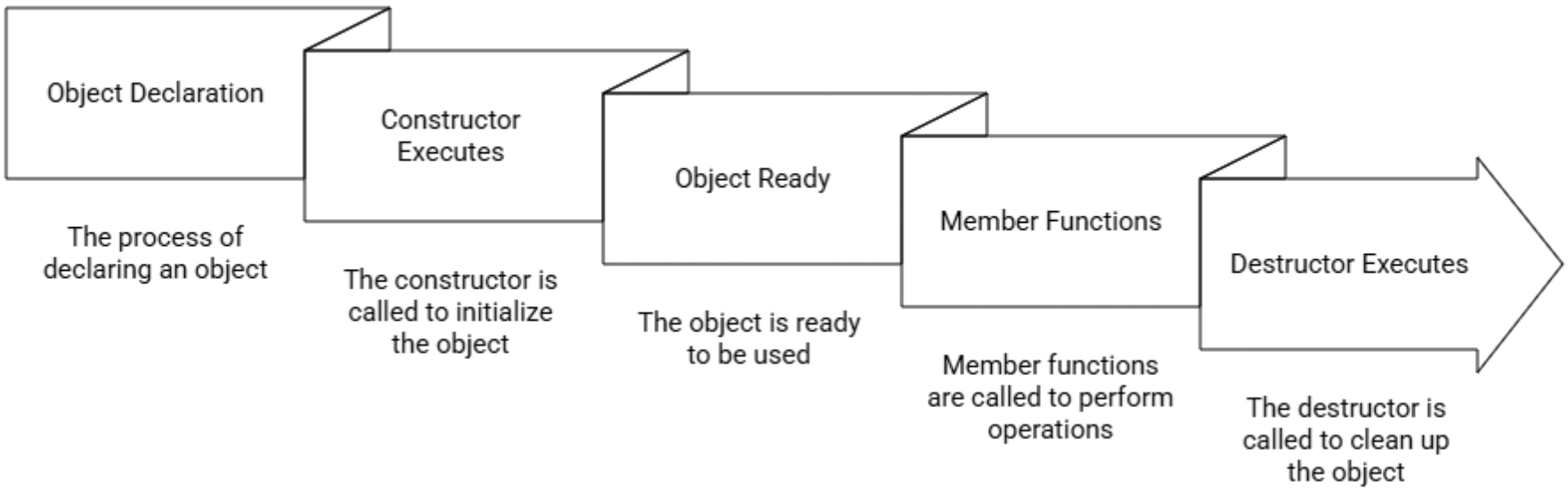
As projects become larger …  Someone forgets to initialize one field … initializes the wrong value …  or assumes another programmer already performed the initialization.

## DESIGN INSIGHT

One of the important ideas in OOP is that  *objects should manage their own correctness.*

The fewer responsibilities placed on programmers who ***use*** the class, the fewer opportunities there are for mistakes.  Constructors embody this principle perfectly.

# OBJECT LIFETIME




Before any member function can execute, the object must first be initialized.

That responsibility belongs to the constructor.

## WHAT IS A CONSTRUCTOR?

Special member function  that executes automatically whenever an object is created.

Constructors differ from ordinary member functions :

Exactly the same name as its class.

No return type .

```
class Song
{
public:
    Song()   { }
};
```

Notice that the constructor is named `Song`.

There is no return type before its name.

Their only responsibility is to prepare an object for use.

# OUR FIRST CONSTRUCTOR

```
class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;
public:
    Song()     {
        title = "Unknown";
        artist = "Unknown";
        duration = 0;
        plays = 0;
    }
```

```
    void print();
    void play();
};
```

```
Song firstSong;
```

## CONSTRUCTOR IS EXECUTED AUTOMATICALLY.

Internally the process looks like this:

```
Allocate Memory ->  Constructor -> Initialize Members   -> Object Ready
```

Every `Song` object now begins in a predictable state.

## COMMON MISTAKE

```
Song song;

song.Song();
```

This is illegal.

Constructors cannot be called like ordinary member functions.

## DEFAULT CONSTRUCTORS

A constructor that receives no parameters is called a  default constructor .

```
Song()
{
    title = "Title";
    artist = "Artist";
    duration = 0;
    plays = 0;
}
```

Now this will be  possible

```
Song first;
Song second;
Song third;
```

Each object receives identical initial values.

Although these values do not represent a real song, they are valid and predictable.

## COMPILER-GENERATED CONSTRUCTORS

Suppose we write

```
class Song {
private:
    string title;
    int duration;
};
```

The compiler generates a default constructor. (In case there are no constructors at all)

However, this constructor performs almost no initialization for primitive data members.

For this reason, professional programmers usually write their own constructors whenever a class has meaningful data that should begin in a known state.

## PARAMETERIZED CONSTRUCTORS

Instead of creating an empty song and assigning values later, we can create the object fully initialized.

```
class Song {
private:
    string title;
    string artist;
    int duration;
public:
    Song(string t,   string a,      int d) {
        title = t;
        artist = a;
        duration = d;
    }
};

Song song("Imagine",          "John Lennon",          183);
```

Now object creation has become much simpler.

## INITIALIZATION VS. MODIFICATION

Compare these two approaches.

Setters : modify an object after it already exists.

```
Song song;

song.setTitle("Imagine");
song.setArtist("John Lennon");
song.setDuration(183);
```

Constructor: initializes the object once

```
Song song("Imagine", "John Lennon", 183);
```

Understanding this distinction is essential.

## CONSTRUCTOR OVERLOADING

Sometimes we want several different ways to create an object.

```
class Song
{
public:
    Song();

    Song(string title);

    Song(string title, string artist, int duration);
};
```

Now programmers can choose whichever constructor best fits the available information.

```
Song sng1;
Song sng2("Imagine");
Song sng3("Imagine",   "John Lennon",        183);
```

# THE `THIS` POINTER

Constructor parameters can  have the same names as data members.

```
Song(string title) {
    title = title;
}
```

Both names refer to the parameter.  The object's data member is never modified.

```
Song(string title) {
    this->title = title;
}
```

- `title` refers to the parameter.
- `this->title` refers to the object's data member.

Using `this` makes the assignment explicit and avoids ambiguity.

## VALIDATING CONSTRUCTOR PARAMETERS

Constructors should also protect the object from invalid data.

```
Song(string t,      string a,      int d)
{
    title = t;
    artist = a;

    if (d < 0)
        duration = 0;
    else
        duration = d;
}
```

Now every song object contains a valid duration.

The programmer using the class cannot accidentally create an impossible object.

## CONSTRUCTORS AND ENCAPSULATION

Constructors strengthen encapsulation: we can restrict incorrect object creation

```
class Song
{
public:
    Song() = delete;

    Song(string title);

    Song(string title, string artist, int duration);
};

Song song; // compilation error!
```

Constructors guarantee that every object starts in a consistent state.

The class becomes responsible for its own correctness.

## SUMMARY

Constructors solve one of the fundamental problems of software design.

Objects should never begin their lives in an undefined or incomplete state.

Instead, every object should be fully initialized automatically when it is created.

We learned that constructors:

- execute automatically,
- have the same name as the class,
- have no return type,
- initialize object data,
- may receive parameters,
- may be overloaded,
- support encapsulation,
- validate object state.

By moving initialization into the class itself, constructors reduce programmer errors and improve software reliability.

## REVIEW QUESTIONS

1. What is a constructor?
2. Why doesn't a constructor have a return type?
3. When is a constructor executed?
4. What is a default constructor?
5. What is a parameterized constructor?
6. Why is constructor overloading useful?
7. What is the difference between initialization and modification?
8. What is the purpose of the `this` pointer?
9. Why should constructors validate parameters?
10. How do constructors support encapsulation?

## LOOKING AHEAD

Throughout this unit we focused on the beginning of an object's lifetime.

Every object is created, initialized, and prepared for use through its constructor. Once initialized, the object performs useful work by responding to member function calls and maintaining its internal state.

However, objects do not exist forever.

Eventually every object reaches the end of its lifetime. Sometimes this simply means releasing the memory occupied by the object. In other situations, an object must free dynamically allocated memory, close files, release network resources, or perform other cleanup tasks before it disappears.

Just as constructors guarantee that every object begins life correctly, C++ provides another special member function to ensure that every object ends its life correctly.

In the next unit we will study  destructors . We will learn when they execute, why they are necessary, and how proper cleanup is just as important as proper initialization. This will prepare us for dynamic memory management and, later in the course, the Rule of Three.